

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
«ВВЕДЕНИЕ В СУБД MONGODB. РАБОТА С БД В СУБД
MONGODB»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Сигаева Ксения Леонидовна
Факультет прикладной информатики
Группа К3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург
2025/2026

Цель работы

Овладеть практическими навыками установки СУБД MongoDB, создания базы данных и коллекций, выполнения CRUD-операций, работы с вложенными объектами, ссылками, индексами и анализом выполнения запросов.

Практическое задание

Установить и проверить работоспособность MongoDB. Создать базу данных learn, заполнить коллекции документами, выполнить выборки, модификацию и удаление документов. Создать документы с вложенными объектами, выполнить агрегацию, задать ссылки между коллекциями, создать индексы и сравнить выполнение запросов с использованием explain.

Выполнение

Практическое задание 2.1.1:

Создание базы данных learn

```
use learn
db.stats()
show collections
```

```
> db.stats()
< {
  db: 'learn',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
> show collections
<
learn>
```

Создание и заполнение коллекции unicorns

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984,
gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender:
'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,
gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601,
gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650,
gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,
gender: 'f'});

db.unicorns.insertOne({name: 'Dunx', loves: ['grape','watermelon'], weight: 704,
gender: 'm', vampires: 165})
db.unicorns.find()
```

```
> db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
< DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a14567f50c97995db38f548')
  }
}
```

```
document=({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender:
'm', vampires: 165})
db. unicorns.insert(document)
db.unicorns.find()
```

```
> document={name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
db.unicorns.insert(document)
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a14578950c97995db38f549')
  }
}
```

```
db.unicorns.find()
```

```
> db.unicorns.find()
< {
  _id: ObjectId('6a14567f50c97995db38f53e'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  _id: ObjectId('6a14567f50c97995db38f53f'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a14567f50c97995db38f540'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ]
}
```

```

    'apple',
    'watermelon'
  ],
  weight: 650,
  gender: 'm',
  vampires: 54
}
{
  _id: ObjectId('6a14567f50c97995db38f548'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a14578950c97995db38f549'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}

```

Практическое задание 2.2.1:

1. Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

Самцы

```

db.unicorns.find(
  {gender: 'm'},
  {name: 1, gender: 1, weight: 1, loves: 1}
).sort({name: 1})

```

```
> db.unicorns.find(
  {gender: 'm'},
  {name: 1, gender: 1, weight: 1, loves: 1}
).sort({name: 1})

< {
  _id: ObjectId('6a14578950c97995db38f549'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm'
}
{
  _id: ObjectId('6a14567f50c97995db38f53e'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm'
}
{
  _id: ObjectId('6a14567f50c97995db38f544'),
  name: 'Kenny',
  loves: [
```

Самки

```
db.unicorns.find(  
  {gender: 'f'},  
  {name: 1, gender: 1, weight: 1, loves: 1}  
).sort({name: 1}).limit(3)
```

```
> db.unicorns.find(  
  {gender: 'f'},  
  {name: 1, gender: 1, weight: 1, loves: 1}  
).sort({name: 1}).limit(3)  
< {  
  _id: ObjectId('6a14567f50c97995db38f53f'),  
  name: 'Aurora',  
  loves: [  
    'carrot',  
    'grape'  
  ],  
  weight: 450,  
  gender: 'f'  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f543'),  
  name: 'Ayna',  
  loves: [  
    'strawberry',  
    'lemon'  
  ],  
  weight: 733,  
  gender: 'f'  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f546'),  
  name: 'Leia',  
  loves: [  
    'apple',
```

2. Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций *findOne* и *limit*.

```
db.unicorns.find(  
  {gender: 'f', loves: 'carrot'},  
  {name: 1, gender: 1, loves: 1, weight: 1}
```

```
)
```

```
> db.unicorns.find(
  {gender: 'f', loves: 'carrot'},
  {name: 1, gender: 1, loves: 1, weight: 1}
)
< {
  _id: ObjectId('6a14567f50c97995db38f53f'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f'
}
{
  _id: ObjectId('6a14567f50c97995db38f542'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f'
}
{
  _id: ObjectId('6a14567f50c97995db38f548'),
  name: 'Nimue',
  loves: [
```

```
db.unicorns.findOne(
  {gender: 'f', loves: 'carrot'},
  {name: 1, gender: 1, loves: 1, weight: 1}
)
```



```

> db.unicorns.findOne(
  {gender: 'f', loves: 'carrot'},
  {name: 1, gender: 1, loves: 1, weight: 1}
)
< {
  _id: ObjectId('6a14567f50c97995db38f53f'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f'
}

```

```

db.unicorns.find(
  {gender: 'f', loves: 'carrot'},
  {name: 1, gender: 1, loves: 1, weight: 1}
).limit(1)

```

```

> db.unicorns.find(
  {gender: 'f', loves: 'carrot'},
  {name: 1, gender: 1, loves: 1, weight: 1}
).limit(1)
< {
  _id: ObjectId('6a14567f50c97995db38f53f'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f'
}

```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
db.unicorns.find(  
  {gender: 'm'},  
  {loves: 0, gender: 0}  
).sort({name: 1})
```

```
> db.unicorns.find(  
  {gender: 'm'},  
  {loves: 0, gender: 0}  
).sort({name: 1})  
< {  
  _id: ObjectId('6a14578950c97995db38f549'),  
  name: 'Dunx',  
  weight: 704,  
  vampires: 165  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f53e'),  
  name: 'Horny',  
  weight: 600,  
  vampires: 63  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f544'),  
  name: 'Kenny',  
  weight: 690,  
  vampires: 39  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f547'),  
  name: 'Pilot',  
  weight: 650,  
  vampires: 54  
}
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({ $natural: -1 })
```

```
> db.unicorns.find().sort({ $natural: -1 })
< {
  _id: ObjectId('6a14578950c97995db38f549'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a14567f50c97995db38f548'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{
  _id: ObjectId('6a14567f50c97995db38f547'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon'
  ],
}
```

Практическое задание 2.1.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find(
  {},
  {
    _id: 0,
    name: 1,
    loves: {$slice: 1}
  }
)
```

```
< {
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ]
}
{
  name: 'Unicrom',
  loves: [
    'energon'
  ]
}
{
  name: 'Roooooodles',
  loves: [
    'apple'
  ]
}
{
  name: 'Solnara',
  loves: [
    'apple'
  ]
}
```

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

```
db.unicorns.find(
  {
    gender: 'f',
```

```
weight: {$gte: 500, $lte: 700}
},
{
  _id: 0
}
)
```

```
> db.unicorns.find(
  {
    gender: 'f',
    weight: {$gte: 500, $lte: 700}
  },
  {
    _id: 0
  }
)
< {
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

```
db.unicorns.find(
  {
```

```
gender: 'm',
weight: {$gte: 500},
loves: {$all: ['grape', 'lemon']}
},
{_id: 0}
)
```

```
> db.unicorns.find(
  {
    gender: 'm',
    weight: {$gte: 500},
    loves: {$all: ['grape', 'lemon']}
  },
  {_id: 0}
)
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

```
db.unicorns.find(
  {vampires: {$exists: false}},
)
```

```

> db.unicorns.find(
  {vampires: {$exists: false}},
)
< {
  _id: ObjectId('6a14567f50c97995db38f548'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}

```

Практическое задание 2.3.4:

Вывести упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```

db.unicorns.find(
  {gender: 'm'},
  { name: 1, loves: {$slice: 1}}
).sort({name: 1})

```

```

> db.unicorns.find(
  {gender: 'm'},
  { name: 1, loves: {$slice: 1}}
).sort({name: 1})
< {
  _id: ObjectId('6a14578950c97995db38f549'),
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  _id: ObjectId('6a14567f50c97995db38f53e'),
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  _id: ObjectId('6a14567f50c97995db38f544'),
  name: 'Kenny',
  loves: [
    'grape'
  ]
}
{
  _id: ObjectId('6a14567f50c97995db38f547'),
  name: 'Pilot',

```

Практическое задание 3.1.1:

1) Создайте коллекцию *towns*, включающую следующие документы:

Создать коллекцию *towns* с вложенными объектами и сформировать запросы по сведениям о мэрах.

```
db.towns.insertMany([
  {name: 'Punxsutawney', population: 6200, last_census: ISODate('2008-01-31'),
    famous_for: ['phil the groundhog'], mayor: {name: 'Jim Wehrle'}},
  {name: 'New York', population: 22200000, last_census: ISODate('2009-07-31'),
    famous_for: ['statue of liberty', 'food'], mayor: {name: 'Michael Bloomberg',
    party: 'I'}},
  {name: 'Portland', population: 528000, last_census: ISODate('2009-07-20'),
    famous_for: ['beer', 'food'], mayor: {name: 'Sam Adams', party: 'D'}}
])
```

```
> use towns
< switched to db towns
> db.towns.insertMany([
  {name: 'Punxsutawney', population: 6200, last_census: ISODate('2008-01-31'), famous_for: ['phil the groundhog'], may
  {name: 'New York', population: 22200000, last_census: ISODate('2009-07-31'), famous_for: ['statue of liberty', 'food
  {name: 'Portland', population: 528000, last_census: ISODate('2009-07-20'), famous_for: ['beer', 'food'], mayor: {nam
  ]})

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a14669b50c97995db38f54a'),
    '1': ObjectId('6a14669b50c97995db38f54b'),
    '2': ObjectId('6a14669b50c97995db38f54c')
  }
}
```

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

```
db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
```

```
> db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
< {
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
```


3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

```
db.towns.find({'mayor.party': {$exists: false}}, {_id: 0, name: 1, mayor: 1})
> db.towns.find({'mayor.party': {$exists: false}}, {_id: 0, name: 1, mayor: 1})
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
```

Практическое задание 3.1.2:

1. Сформировать функцию для вывода списка самцов единорогов.

```
fn = function() {
  return this.gender == 'm';
}
```

```
> fn = function() {
  return this.gender == 'm';
}
< [Function: fn]
```

2. Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.

```
var cursor = db.unicorns.find(
  {$where: fn},
  {_id: 0, name: 1, gender: 1}
).sort({name: 1}).limit(2);
```

3. Вывести результат, используя *forEach*.

```
cursor.forEach(function(obj) {
  print(obj.name);
})
```

```
> cursor.forEach(function(obj) {
  print(obj.name);
})
< Dunx
< Horny
```

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

```
db.unicorns.find({
  gender: 'f',
  weight: {$gte: 500, $lte: 600}
}).count()
```

```
> db.unicorns.find({
  gender: 'f',
  weight: {$gte: 500, $lte: 600}
}).count()
< 2
```

Практическое задание 3.2.2:

Вывести список предпочтений.

```
db.unicorns.distinct('loves')
```

```
> db.unicorns.distinct('loves')
< [
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

```
db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: {$sum: 1}
    }
  }
])
```

```
> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: {$sum: 1}
    }
  }
])
< {
  _id: 'm',
  count: 7
}
{
  _id: 'f',
  count: 5
}
```

Практическое задание 3.3.1:

1. Выполнить команду:

```
db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})

db.unicorns.insertOne({
  name: 'Barney',
  loves: ['grape'],
  weight: 340,
  gender: 'm'
})
```

2. Проверить содержимое коллекции unicorns.

```
db.unicorns.find({name: 'Barney'})
```

```

> db.unicorns.find({name: 'Barney'})
< {
  _id: ObjectId('6a146bbe50c97995db38f54d'),
  name: 'Barney',
  loves: [
    'grape'
  ],
  weight: 340,
  gender: 'm'
}

```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения: теперь ее вес 800, она убила 51 вампира.

```

db.unicorns.update(
  {name: 'Ayna'},
  {$set: {weight: 800, vampires: 51}}
)

```

2. Проверить содержимое коллекции unicorns.

```

db.unicorns.find({name: 'Ayna'})
> db.unicorns.find({name: 'Ayna'})
< {
  _id: ObjectId('6a14567f50c97995db38f543'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}

```

Практическое задание 3.3.3:

Для самца единорога Raleigh внести изменения: теперь он любит redbull.

```
db.unicorns.updateOne(  
  {name: 'Raleigh'},  
  {$set: {loves: ['redbull']}}  
)  
db.unicorns.find({name: 'Raleigh'})
```

```
> db.unicorns.find({name: 'Raleigh'})  
< {  
  _id: ObjectId('6a14567f50c97995db38f545'),  
  name: 'Raleigh',  
  loves: [  
    'redbull'  
  ],  
  weight: 421,  
  gender: 'm',  
  vampires: 2  
}
```

Практическое задание 3.3.4:

Всем самцам единорогов увеличить количество убитых вампиров на 5 и проверить коллекцию.

```
db.unicorns.updateMany(  
  {gender: 'm'},  
  {$inc: {vampires: 5}}  
)  
db.unicorns.find({gender: 'm'})
```

```
db.unicorns.find({gender: 'm'})
```

```
< {
  _id: ObjectId('6a14567f50c97995db38f53e'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
{
  _id: ObjectId('6a14567f50c97995db38f540'),
  name: 'Unicrom',
  loves: [
    'energon',
    'redbull'
  ],
  weight: 984,
  gender: 'm',
  vampires: 187
}
{
  _id: ObjectId('6a14567f50c97995db38f541'),
  name: 'Roooooodles',
  loves: [
    'apple'
  ],
  weight: 1000,
  gender: 'f',
  vampires: 100
}
```

Практическое задание 3.3.5:

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

```
db.towns.updateOne(  
  {name: 'Portland'},  
  {$unset: {'mayor.party': 1}}  
)  
db.towns.find({name: 'Portland'})
```

```
> db.towns.find({name: 'Portland'})  
< {  
  _id: ObjectId('6a14669b50c97995db38f54c'),  
  name: 'Portland',  
  population: 528000,  
  last_census: 2009-07-20T00:00:00.000Z,  
  famous_for: [  
    'beer',  
    'food'  
  ],  
  mayor: {  
    name: 'Sam Adams'  
  }  
}
```

Практическое задание 3.3.6:

Изменить информацию о самце единорога Pilot: теперь он любит и chocolate.

```
db.unicorns.updateOne(  
  {name: 'Pilot'},  
  {$push: {'likes': 'chocolate'}}  
)
```

```
{name: 'Pilot'},  
{$push: {loves: 'chocolate'}}}  
)  
db.unicorns.find({name: 'Pilot'})
```

```
db.unicorns.find({name: 'Pilot'})  
  
< {  
  _id: ObjectId('6a14567f50c97995db38f547'),  
  name: 'Pilot',  
  loves: [  
    'apple',  
    'watermelon',  
    'chocolate'  
  ],  
  weight: 650,  
  gender: 'm',  
  vampires: 59  
}
```

Практическое задание 3.3.7:

Изменить информацию о самке единорога Aurora: теперь она любит еще sugar и watermelon.

```
db.unicorns.updateOne(  
  {name: 'Aurora'},  
  {$addToSet: {loves: {$each: ['sugar', 'watermelon']}}}  
)  
db.unicorns.find({name: 'Aurora'})
```



```
db.unicorns.find({name: 'Aurora'})  
  
< {  
  _id: ObjectId('6a14567f50c97995db38f53f'),  
  name: 'Aurora',  
  loves: [  
    'carrot',  
    'grape',  
    'sugar',  
    'watermelon'  
  ],  
  weight: 450,  
  gender: 'f',  
  vampires: 43  
}
```

Практическое задание 3.4.1:

1. *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",  
popujatiuon: 6200,  
last_sensus: ISODate("2008-01-31"),  
famous_for: ["phil the groundhog"],  
mayor: {  
  name: "Jim Wehrle"  
}}  
{name: "New York",  
popujatiuon: 22200000,  
last_sensus: ISODate("2009-07-31"),  
famous_for: ["status of liberty", "food"],  
mayor: {
```

```

    name: "Michael Bloomberg",
party: "I"}}
{name: "Portland",
popujatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
    name: "Sam Adams",
party: "D"}}

```

```

db.towns.drop()

db.towns.insertMany([
{
    name: "Punxsutawney",
    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
        name: "Jim Wehrle"
    }
},
{
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
        name: "Michael Bloomberg",
        party: "I"
    }
},
{
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
        name: "Sam Adams",
        party: "D"
    }
}
])

```

2. Удалите документы с беспартийными мэрами.

```

db.towns.deleteMany({'mayor.party': {'$exists': false}})

```

```
> db.towns.deleteMany({'mayor.party': {$exists: false}})
< {
  acknowledged: true,
  deletedCount: 1
}
```

3. *Проверьте содержание коллекции.*

```
> db.towns.find()
< {
  _id: ObjectId('6a146e3f50c97995db38f54f'),
  name: 'New York',
  popujatiuon: 22200000,
  last_sensus: 2009-07-31T00:00:00.000Z,
  famous_for: [
    'status of liberty',
    'food'
  ],
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
{
  _id: ObjectId('6a146e3f50c97995db38f550'),
  name: 'Portland',
  popujatiuon: 528000,
  last_sensus: 2009-07-20T00:00:00.000Z,
  famous_for: [
    'beer',
    'food'
  ],
  mayor: {
    name: 'Sam Adams',
    party: 'D'
  }
}
```

4. *Очистите коллекцию.*

```
db.towns.deleteMany({})
```

```
> db.towns.deleteMany({})
< {
  acknowledged: true,
  deletedCount: 2
}
```

5. *Просмотрите список доступных коллекций.*

```
show collections
```

```
> show collections
```

```
< towns
```

Практическое задание 4.1.1:

1. *Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.*

```
db.habitats.insertMany([
  { _id: 'forest', short_name: 'forest', full_name: 'Лесная зона', description:
'Зона обитания около леса'},
  { _id: 'mountains', short_name: 'mountains', full_name: 'Горная зона',
description: 'Зона обитания в горах'},
  { _id: 'field', short_name: 'field', full_name: 'Полевая зона', description:
'Открытая полевая зона'}
])
```

2. *Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.*

```
db.unicorns.updateMany(
  {name: {$in: ['Aurora', 'Nimue', 'Leia']}},
  {$set: {habitat: {$ref: 'habitats', $id: 'forest'}}}
)
```

3. *Проверьте содержание коллекции единорогов.*

```
db.unicorns.find()
```

```
weight: 601,  
gender: 'f',  
vampires: 33,  
habitat: DBRef('habitats', 'forest')  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f547'),  
  name: 'Pilot',  
  loves: [  
    'apple',  
    'watermelon',  
    'chocolate'  
  ],  
  weight: 650,  
  gender: 'm',  
  vampires: 59  
}  
{  
  _id: ObjectId('6a14567f50c97995db38f548'),  
  name: 'Nimue',  
  loves: [  
    'grape',  
    'carrot'  
  ],  
  weight: 540,  
  gender: 'f',  
  habitat: DBRef('habitats', 'forest')  
}
```

Практическое задание 4.2.1:

Проверить, можно ли задать для коллекции unicorns индекс для ключа name с флагом unique.

```
db.unicorns.createIndex({name: 1}, {unique: true})
db.unicorns.getIndexes()
```

```
> db.unicorns.createIndex({name: 1}, {unique: true})
< name_1
```

Практическое задание 4.3.1:

Получить информацию о всех индексах коллекции unicorns, удалить все индексы кроме _id и попытаться удалить индекс _id.

```
db.unicorns.getIndexes()
db.unicorns.dropIndexes()
db.unicorns.getIndexes()
db.unicorns.dropIndex ('_id_')
```

```
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

```
> db.unicorns.dropIndexes()
< {
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

```
> db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

```
> db.unicorns.dropIndex ('_id_')
```

```
✖ ▸ MongoServerError[InvalidOptions]: cannot drop _id index
```

Практическое задание 4.4.1:

1. *Создайте объемную коллекцию numbers, задействовав курсор:*

```
for (let i = 0; i < 100000; i++) {  
  db.numbers.insertOne({value: i})  
}
```

2. *Выберите последних четыре документа.*

```
db.numbers.find().sort({$natural: -1}).limit(4).explain("executionStats")
```

3. *Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра executionTimeMillis)*

```
rejectedPlans: []  
},  
executionStats: {  
  executionSuccess: true,  
  nReturned: 4,  
  executionTimeMillis: 4,  
  totalKeysExamined: 0,  
  totalDocsExamined: 4,  
  executionStages: {  
    isCached: false,  
    stage: 'LIMIT',  
    nReturned: 4,
```

4. *Создайте индекс для ключа value.*

```
db.numbers.createIndex({value: 1})
```

5. *Получите информацию о всех индексах коллекции numbers.*

```
db.numbers.getIndexes()
```

```
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

6. *Выполните запрос 2.*

7. *Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?*

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1
  }
}
```

8. *Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?*

Запрос с индексом более эффективен.

Выводы

В ходе лабораторной работы была изучена СУБД MongoDB как документно-ориентированная NoSQL-система. Были выполнены операции создания базы данных и коллекций, вставки, выборки, обновления и удаления документов. Также были рассмотрены вложенные объекты, массивы, ссылки между коллекциями, агрегирующие запросы, создание индексов и анализ планов выполнения запросов.

Практическая часть показала, что MongoDB удобно использовать для хранения гибких документов без жесткой табличной схемы, а индексы позволяют ускорять выборку данных при работе с большими коллекциями.